

CM0859 – MT5009 Type Theory

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Andrés Sicard-Ramírez

Universidad EAFIT

Semester 2025-2

Preliminaries

Textbook

Pfenning (2023). Lecture Notes on Proofs as Programs.

Conventions

The numbers and page numbers assigned to chapters, examples, exercises, figures, quotes, sections and theorems on these slides correspond to the numbers assigned in the textbook.

Source code

The AGDA code have been tested with AGDA 2.8.0.

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Introduction

- Propositions (types): $A, B, C, \dots$

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Introduction

- Propositions (types): $A, B, C, \dots$
- Lambda terms (programs): $M, N, \dots$

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Introduction

- Propositions (types): $A, B, C, \dots$
- Lambda terms (programs): $M, N, \dots$
- Judgements:
 - (i) $A \text{ type}$ (A is a type)
 - (ii) $M : A$ (λ -term M is a proof-term for proposition A)
(program M has type A)
 - (iii) $M \Rightarrow_R M'$ (λ -term M reduces to λ -term M')

(continued on next slide)

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Introduction (continuation)

- Proposition-as-types principle

“We intend that if $M : A$ then A true. Conversely, if A true then $M : A$ for some appropriate proof-term M . But we want something more: every deduction of $M : A$ should correspond to a deduction of A true with an identical structure and vice versa.” (p. L4.1)

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Introduction (continuation)

- Proposition-as-types principle

“We intend that if $M : A$ then A true. Conversely, if A true then $M : A$ for some appropriate proof-term M . But we want something more: every deduction of $M : A$ should correspond to a deduction of A true with an identical structure and vice versa.” (p. L4.1)

*“The introduction rules for a logical connective correspond to the **constructors** for the corresponding type. Conversely, the elimination rules correspond to **destructors**.” (p. L4.2)*

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Introduction (continuation)

- Proposition-as-types principle

“We intend that if $M : A$ then A true. Conversely, if A true then $M : A$ for some appropriate proof-term M . But we want something more: every deduction of $M : A$ should correspond to a deduction of A true with an identical structure and vice versa.” (p. L4.1)

*“The introduction rules for a logical connective correspond to the **constructors** for the corresponding type. Conversely, the elimination rules correspond to **destructors**.” (p. L4.2)*

- Form of the inference rules (same than for natural deduction):

$$\frac{J_1 \quad \dots \quad J_n}{J} \text{ rule name}$$

where J (conclusion) and $J_1, \dots, J_n$ (premises or hypotheses) are all judgements.

(continued on next slide)

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Introduction (continuation)

- Inference rules (same than for natural deduction): Formation, introduction and elimination rules.

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Introduction (continuation)

- Inference rules (same than for natural deduction): Formation, introduction and elimination rules.
- New inference rules: Reduction rules.

“If proofs are programs then we need to explain how proofs are to be executed, and which results may be returned by a computation.”

“Reduction arises when a destructor is applied to a constructor.”

“A proof reduction arises when we apply an elimination rule to the result of an introduction rules.” (p. L4.7)

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Introduction (continuation)

- Inference rules (same than for natural deduction): Formation, introduction and elimination rules.
- New inference rules: Reduction rules.[†]

“If proofs are programs then we need to explain how proofs are to be executed, and which results may be returned by a computation.”

“Reduction arises when a destructor is applied to a constructor.”

“A proof reduction arises when we apply an elimination rule to the result of an introduction rules.” (p. L4.7)

[†]Some authors introduce *computation* rules instead of *reduction* rules.

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Observation

Other names for the proposition-as-types principle are (or should be):

- the Curry-Howard correspondence,
- the Curry-Howard isomorphism (textbook),
- the Brouwer-Heyting-Kolmogorov-Schönfinkel-Curry-Meredith-Kleene-Feys-Gödel-Läuchli-Kreisel-Tait-Lawvere-Howard-de Bruijn-Scott-Martin Löf-Girard-Reynolds-Stenlund-Constable-Coquand-Huet... correspondence (Sørensen and Urzyczyn 2006, p. viii).

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Definition

The **types** of typed lambda calculus are defined by the following grammar:

$\sigma, \tau ::= \mathbf{b}$	(base type)
$\sigma \rightarrow \tau$	(function type)
$\sigma \times \tau$	(product type)
$\sigma + \tau$	((disjoint) sum type or coproduct type)
$\mathbf{1}$	(unit type)
$\mathbf{0}$	(empty type)

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Definition

The λ -**terms** of our typed lambda calculus are defined by the following grammar:

$M, N, P ::= x$	(variable)
$\lambda x.M$	(λ -abstraction)
$M N$	(application)
$\langle M, N \rangle$ $\text{fst } M$ $\text{snd } M$	(pair and projections)
$\text{inl } M$ $\text{inr } M$ $\text{case}(M, x.N, y.P)$	(injections and case analysis)
$\langle \rangle$	(unit)
$\text{abort } M$	(error)

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Inference rules for conjunction (product type)

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Inference rules for conjunction (product type)

- Formation rule

$$\frac{A_{\text{prop}} \quad B_{\text{prop}}}{A \wedge B_{\text{prop}}} \wedge F \qquad \frac{A_{\text{type}} \quad B_{\text{type}}}{A \wedge B_{\text{type.}}} \wedge F$$

(continued on next slide)

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Inference rules for conjunction (product type) (continuation)

- Introduction rule

$$\frac{A \text{ true} \quad B \text{ true}}{A \wedge B \text{ true},} \wedge I \qquad \frac{M : A \quad N : B}{\langle M, N \rangle : A \wedge B.} \wedge I$$

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Inference rules for conjunction (product type) (continuation)

- Introduction rule

$$\frac{A \text{ true} \quad B \text{ true}}{A \wedge B \text{ true},} \wedge I$$

$$\frac{M : A \quad N : B}{\langle M, N \rangle : A \wedge B.} \wedge I$$

- Elimination rules

$$\frac{A \wedge B \text{ true}}{A \text{ true},} \wedge E_1$$

$$\frac{M : A \wedge B}{\text{fst } M : A,} \wedge E_1$$

$$\frac{A \wedge B \text{ true}}{B \text{ true},} \wedge E_2$$

$$\frac{M : A \wedge B}{\text{snd } M : B.} \wedge E_2$$

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Inference rules for conjunction (product type) (continuation)

- Introduction rule

$$\frac{A \text{ true} \quad B \text{ true}}{A \wedge B \text{ true},} \wedge I \qquad \frac{M : A \quad N : B}{\langle M, N \rangle : A \wedge B.} \wedge I$$

- Elimination rules

$$\frac{A \wedge B \text{ true}}{A \text{ true},} \wedge E_1 \qquad \frac{M : A \wedge B}{\text{fst } M : A,} \wedge E_1$$
$$\frac{A \wedge B \text{ true}}{B \text{ true},} \wedge E_2 \qquad \frac{M : A \wedge B}{\text{snd } M : B.} \wedge E_2$$

- Reduction rules

$$\text{fst } \langle M, N \rangle \Rightarrow_R M,$$
$$\text{snd } \langle M, N \rangle \Rightarrow_R N.$$

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Inference rules for implication (function type)

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Inference rules for implication (function type)

- Formation rule

$$\frac{A \text{ prop} \quad B \text{ prop}}{A \supset B \text{ prop},} \supset F \qquad \frac{A \text{ type} \quad B \text{ type}}{A \supset B \text{ type}.} \supset F$$

(continued on next slide)

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Inference rules for implication (function type) (continuation)

- Introduction rule

$$\frac{\frac{\frac{}{A \text{ true}}^u \quad \vdots}{B \text{ true}}}{A \supset B \text{ true},} \supset I^u \qquad \frac{\frac{\frac{}{u : A}}{M : B}^u \quad \vdots}{\lambda u. M : A \supset B.} \supset I^u$$

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Inference rules for implication (function type) (continuation)

- Introduction rule

$$\begin{array}{c} \frac{}{A \text{ true}}^u \\ \vdots \\ \frac{B \text{ true}}{A \supset B \text{ true},} \supset I^u \end{array} \qquad \begin{array}{c} \frac{}{u : A}^u \\ \vdots \\ \frac{M : B}{\lambda u. M : A \supset B.} \supset I^u \end{array}$$

- Elimination rule

$$\frac{A \supset B \text{ true} \quad A \text{ true}}{B \text{ true},} \supset E \qquad \frac{M : A \supset B \quad N : A}{M N : B.} \supset E$$

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Inference rules for implication (function type) (continuation)

- Introduction rule

$$\begin{array}{c} \frac{}{A \text{ true}}^u \\ \vdots \\ \frac{B \text{ true}}{A \supset B \text{ true},} \supset I^u \end{array} \qquad \begin{array}{c} \frac{}{u : A}^u \\ \vdots \\ \frac{M : B}{\lambda u. M : A \supset B.} \supset I^u \end{array}$$

- Elimination rule

$$\frac{A \supset B \text{ true} \quad A \text{ true}}{B \text{ true},} \supset E \qquad \frac{M : A \supset B \quad N : A}{M N : B.} \supset E$$

- Reduction rule

$$(\lambda u. M) N \Rightarrow_R M [u := N].$$

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Observation

For annotating a proof (derivation) with proof terms we proceed top-down as illustrated by the following examples.

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Example

(i) A proof that $A \supset A$ true.

$$\frac{\frac{}{A \text{ true}}^u}{A \supset A \text{ true}} \supset I^u$$

(ii) A proof that $\lambda u. u : A \supset A$.

$$\frac{\frac{}{u : A}^u}{\lambda u. u : A \supset A} \supset I^u$$

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Example

(i) A proof that $(A \wedge B) \supset (B \wedge A)$ true.

$$\frac{\frac{\frac{}{x} \quad A \wedge B \text{ true}}{B \text{ true}} \wedge E_2 \quad \frac{\frac{}{x} \quad A \wedge B \text{ true}}{A \text{ true}} \wedge E_1}{B \wedge A \text{ true}} \wedge I}{(A \wedge B) \supset (B \wedge A) \text{ true}} \supset I^x$$

(ii) A proof that $\lambda x. \langle \text{snd } x, \text{fst } x \rangle : (A \wedge B) \supset (B \wedge A)$.

$$\frac{\frac{\frac{}{x} \quad x : A \wedge B}{\text{snd } x : B} \wedge E_2 \quad \frac{\frac{}{x} \quad x : A \wedge B}{\text{fst } x : A} \wedge E_1}{\langle \text{snd } x, \text{fst } x \rangle : B \wedge A} \wedge I}{\lambda x. \langle \text{snd } x, \text{fst } x \rangle : (A \wedge B) \supset (B \wedge A)} \supset I^x$$

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Example

(i) A proof that $A \supset (B \supset A)$ true.

$$\frac{\frac{\frac{}{u} \quad A \text{ true}}{B \supset A \text{ true}} \supset I}{A \supset (B \supset A) \text{ true}} \supset I^u$$

(ii) A proof that $\lambda u. \lambda w. u : A \supset (B \supset A)$.

$$\frac{\frac{\frac{}{u} \quad u : A}{\lambda w. u : B \supset A} \supset I}{\lambda u. \lambda w. u : A \supset (B \supset A)} \supset I^u$$

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Notation

Let Γ be a set of judgements and let $M : A$ be a judgement. The relation $\Gamma \vdash M : A$ means that there is a derivation with conclusion $M : A$ from the set of hypotheses Γ .

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Example

(i) A proof that $A \wedge B \text{ true} \vdash B \wedge A \text{ true}$.

$$\frac{\frac{A \wedge B \text{ true}}{B \text{ true}} \wedge E_2 \quad \frac{A \wedge B \text{ true}}{A \text{ true}} \wedge E_1}{B \wedge A \text{ true}} \wedge I$$

(ii) A proof that $M : A \wedge B \vdash \langle \text{snd } M, \text{fst } M \rangle : B \wedge A$.

$$\frac{\frac{M : A \wedge B}{\text{snd } M : B} \wedge E_2 \quad \frac{M : A \wedge B}{\text{fst } M : A} \wedge E_1}{\langle \text{snd } M, \text{fst } M \rangle : B \wedge A} \wedge I$$

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Inference rules for disjunction (sum type or coproduct type)

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Inference rules for disjunction (sum type or coproduct type)

- Formation rule

$$\frac{A_{\text{prop}} \quad B_{\text{prop}}}{A \vee B_{\text{prop},}} \vee F \qquad \frac{A_{\text{type}} \quad B_{\text{type}}}{A \vee B_{\text{type}.}} \vee F$$

(continued on next slide)

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Inference rules for disjunction (sum type or coproduct type) (continuation)

- Introduction rules

$$\begin{array}{c} \frac{A \text{ true}}{A \vee B \text{ true},} \vee I_1 \qquad \frac{M : A}{\text{inl } M : A \vee B.} \vee I_1 \\[1em] \frac{B \text{ true}}{A \vee B \text{ true},} \vee I_2 \qquad \frac{N : B}{\text{inr } N : A \vee B.} \vee I_2 \end{array}$$

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Inference rules for disjunction (sum type or coproduct type) (continuation)

- Introduction rules

$$\frac{A \text{ true}}{A \vee B \text{ true},} \vee I_1 \qquad \frac{M : A}{\text{inl } M : A \vee B.} \vee I_1$$

$$\frac{B \text{ true}}{A \vee B \text{ true},} \vee I_2 \qquad \frac{N : B}{\text{inr } N : A \vee B.} \vee I_2$$

- Elimination rule

$$\frac{\frac{\frac{}{A \text{ true}}^u \quad \frac{}{B \text{ true}}^w}{\vdots} \quad \frac{}{C \text{ true}}}{A \vee B \text{ true} \quad C \text{ true},} \vee E^{u,w} \qquad \frac{\frac{\frac{}{u : A}^u \quad \frac{}{w : B}^w}{\vdots} \quad \frac{}{P : C}}{M : A \vee B \quad N : C \quad P : C} \vee E^{u,w}}{\text{case}(M, u.N, w.P) : C.}$$

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Inference rules for disjunction (sum type or coproduct type) (continuation)

- Introduction rules

$$\frac{A \text{ true}}{A \vee B \text{ true},} \vee I_1 \qquad \frac{M : A}{\text{inl } M : A \vee B.} \vee I_1$$

$$\frac{B \text{ true}}{A \vee B \text{ true},} \vee I_2 \qquad \frac{N : B}{\text{inr } N : A \vee B.} \vee I_2$$

- Elimination rule

$$\frac{\frac{\frac{}{A \text{ true}} \quad u \quad \frac{}{B \text{ true}} \quad w}{\vdots \quad \vdots} \quad \frac{A \vee B \text{ true} \quad C \text{ true} \quad C \text{ true}}{C \text{ true},} \vee E^{u,w}}{\frac{M : A \vee B \quad N : C \quad P : C}{\text{case}(M, u.N, w.P) : C.} \vee E^{u,w}}$$

- Reduction rules

$$\text{case}(\text{inl } M, u.N, w.P) \Rightarrow_R N [u := M],$$

$$\text{case}(\text{inr } M, u.N, w.P) \Rightarrow_R P [w := M].$$

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Example

(i) A proof that $(A \vee B) \supset (B \vee A)$ true.

$$\frac{\frac{\frac{}{A \vee B \text{ true}}^u \quad \frac{\frac{}{A \text{ true}}^v}{B \vee A \text{ true}} \vee I_2 \quad \frac{\frac{}{B \text{ true}}^w}{B \vee A \text{ true}} \vee I_1}{B \vee A \text{ true}} \vee E^{v,w}}{(A \vee B) \supset (B \vee A) \text{ true}} \supset I^u$$

(ii) A proof that $\lambda u. \text{case}(u, v. \text{inr } v, w. \text{inl } w) : (A \vee B) \supset (B \vee A)$.

$$\frac{\frac{\frac{}{u : A \vee B}^u \quad \frac{\frac{}{v : A}}{\text{inr } v : B \vee A} \vee I_2 \quad \frac{\frac{}{w : B}}{\text{inl } w : B \vee A} \vee I_1}{\text{case}(u, v. \text{inr } v, w. \text{inl } w) : B \vee A} \vee E^{v,w}}{\lambda u. \text{case}(u, v. \text{inr } v, w. \text{inl } w) : (A \vee B) \supset (B \vee A)} \supset I^u$$

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Inference rules for truth (unit type)

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Inference rules for truth (unit type)

- Formation rule

$$\frac{}{\top_{\text{prop}}} \top F$$

$$\frac{}{\top_{\text{type}}} \top F$$

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Inference rules for truth (unit type)

- Formation rule

$$\frac{}{\top_{\text{prop}}} \top F$$

$$\frac{}{\top_{\text{type}}} \top F$$

- Introduction rule

$$\frac{}{\top_{\text{true}}} \top I$$

$$\frac{}{\langle \rangle : \top} \top I$$

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Inference rules for truth (unit type)

- Formation rule

$$\frac{}{\top_{\text{prop}}} \top F$$

$$\frac{}{\top_{\text{type}}} \top F$$

- Introduction rule

$$\frac{}{\top_{\text{true}}} \top I$$

$$\frac{}{\langle \rangle : \top} \top I$$

- No elimination rule
- No reduction rule

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Inference rules for falsehood (empty type)

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Inference rules for falsehood (empty type)

- Formation rule

$$\frac{}{\perp_{\text{prop}}, \perp F}$$

$$\frac{}{\perp_{\text{type.}}, \perp F}$$

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Inference rules for falsehood (empty type)

- Formation rule

$$\frac{}{\perp \text{ prop,}} \perp F$$

$$\frac{}{\perp \text{ type.}} \perp F$$

- Elimination rule

$$\frac{\perp \text{ true}}{A \text{ true,}} \perp E$$

$$\frac{M : \perp}{\text{abort } M : A.} \perp E$$

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Inference rules for falsehood (empty type)

- Formation rule

$$\frac{}{\perp \text{ prop,}} \perp F$$

$$\frac{}{\perp \text{ type.}} \perp F$$

- Elimination rule

$$\frac{\perp \text{ true}}{A \text{ true,}} \perp E$$

$$\frac{M : \perp}{\text{abort } M : A.} \perp E$$

- No introduction rule
- No reduction rule

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Guiding principles behind the proposition-as-types principle

Textbook (p. L4.6):

- (i) *Every proposition corresponds to a type and vice versa.*
- (ii) *Introduction rules correspond to value constructors.*
- (iii) *Elimination rules correspond to value destructors.*
- (iv) *For every deduction of A **true** there is a proof term M and deduction of $M : A$. We can effectively construct this top-down.*
- (v) *For every deduction of $M : A$ there is a deduction of A **true**. We can effectively construct this bottom-up.*

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Agda implementation of product types (conjunction)

- Formation and introduction rules

$$\frac{A \text{ type} \quad B \text{ type}}{A \times B \text{ type,}} \times F$$

$$\frac{M : A \quad N : B}{\langle M, N \rangle : A \times B.} \times I$$

```
1  data _×_ (A B : Set) : Set where
2  ⟨_,_⟩ : A → B → A × B
```

(continued on next slide)

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Agda implementation of product types (conjunction) (continuation)

- Elimination and reduction rules

$$\frac{M : A \times B}{\text{fst } M : A,} \times E_1$$

$$\frac{M : A \times B}{\text{snd } M : B,} \times E_2$$

$$\begin{aligned} \text{fst } \langle M, N \rangle &\Rightarrow_R M, \\ \text{snd } \langle M, N \rangle &\Rightarrow_R N. \end{aligned}$$

```
1  fst : {A B : Set} → A × B → A
2  fst ⟨ a , _ ⟩ = a
3
4  snd : {A B : Set} → A × B → B
5  snd ⟨ _ , b ⟩ = b
```

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Agda implementation of function types (implication)

The function type $\rightarrow$ is built-in in Agda.

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Example

A proof and an implementation that $\lambda a.a : A \rightarrow A$.

$$\frac{\frac{}{a : A} \quad a}{\lambda a.a : A \rightarrow A} \rightarrow |^a$$

```
1  id1 : {A : Set} → A → A
2  id1 = λ a → a
3
4  id2 : {A : Set} → A → A
5  id2 a = a
```

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Example

A proof and an implementation that $\lambda x. \langle \text{snd } x, \text{fst } x \rangle : (A \times B) \rightarrow (B \times A)$.

$$\frac{\frac{\frac{}{x : A \times B}}{\text{snd } x : B} \times E_2 \quad \frac{\frac{}{x : A \times B}}{\text{fst } x : A} \times E_1}{\langle \text{snd } x, \text{fst } x \rangle : B \times A} \times I}{\lambda x. \langle \text{snd } x, \text{fst } x \rangle : (A \times B) \rightarrow (B \times A)} \rightarrow I^x$$

```
1  ×-comm1 : {A B : Set} → A × B → B × A
2  ×-comm1 = λ x → ⟨ snd x , fst x ⟩
3
4  -- Using pattern-matching
5  ×-comm2 : {A B : Set} → A × B → B × A
6  ×-comm2 ⟨ a , b ⟩ = ⟨ b , a ⟩
```

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Example

A proof and an implementation that $\lambda a. \lambda b. a : A \rightarrow (B \rightarrow A)$.

$$\frac{\frac{\frac{}{a : A} a}{\lambda b. a : B \rightarrow A} \rightarrow I}{\lambda a. \lambda b. a : A \rightarrow (B \rightarrow A)} \rightarrow I^a$$

```
1  ax1 : {A B : Set} → A → (B → A)
2  ax1 = λ a _ → a
3
4  ax2 : {A B : Set} → A → (B → A)
5  ax2 a _ = a
```

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Agda implementation of sum types (disjunction)

- Formation and introduction rules

$$\frac{A \text{ type} \quad B \text{ type}}{A + B \text{ type,}} +F$$

$$\frac{M : A}{\text{inl } M : A + B,} +I_1$$

$$\frac{N : B}{\text{inr } N : A + B.} +I_2$$

```
1  data _+_ (A B : Set) : Set where  
2    inl  : A → A + B  
3    inr  : B → A + B
```

(continued on next slide)

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Agda implementation of sum types (disjunction) (continuation)

- Elimination and reduction rules

$$\frac{\frac{\frac{}{u : A}^u \quad \frac{}{w : B}^w}{\vdots} \quad \frac{M : A + B \quad N : C \quad P : C}{\text{case}(M, u.N, w.P) : C,} +E^{u,w}$$

$\text{case}(\text{inl } M, u.N, w.P) \Rightarrow_R N [u := M],$
 $\text{case}(\text{inr } M, u.N, w.P) \Rightarrow_R P [w := M].$

```
1  case : {A B C : Set} →
2      A + B →
3      (A → C) →
4      (B → C) →
5      C
6  case (inl a) f g = f a
7  case (inr b) f g = g b
```

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Example

A proof and an implementation that $\lambda u. \text{case}(x, a. \text{inr } a, b. \text{inl } b) : (A + B) \rightarrow (B + A)$.

$$\frac{\frac{\frac{}{u : A + B} x \quad \frac{\frac{}{a : A} a}{\text{inr } v : B + A} +l_2 \quad \frac{\frac{}{b : B} b}{\text{inl } b : B + A} +l_1}{\text{case}(x, a. \text{inr } a, b. \text{inl } b) : B + A} +E^{a,b}}{\lambda x. \text{case}(x, a. \text{inr } a, b. \text{inl } b) : (A + B) \rightarrow (B + A)} \rightarrow I^x$$

```
1  +-comm1 : {A B : Set} → A + B → B + A
2  +-comm1 = λ x → case x (λ a → inr a) (λ b → inl b)
3
4  -- Using pattern-matching
5  +-comm2 : {A B : Set} → A + B → B + A
6  +-comm2 (inl a) = inr a
7  +-comm2 (inr b) = inl b
```

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Agda implementation of unit type (truth)

- Formation and introduction rules

$$\frac{}{\top \text{ type,}} \top F$$

$$\frac{\langle \rangle : \top.}{\top I}$$

```
1 data  $\top$  : Set where
2    $\langle \rangle$  :  $\top$ 
```

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Agda implementation of empty type (falsehood)

- Formation rule

$$\frac{}{\perp \text{ type.}} \perp F$$

```
data  $\perp$  : Set where
```


Typed Lambda Calculus (or the Proposition-as-Types Principle)

Agda implementation of empty type (falsehood)

- Formation rule

$$\frac{}{\perp \text{ type.}} \perp F$$

```
data ⊥ : Set where
```

- Elimination rule

$$\frac{M : \perp}{\text{abort } M : A.} \perp E$$

```
1 abort : {A : Set} → ⊥ → A
2 abort ()
```

Typed Lambda Calculus (or the Proposition-as-Types Principle)

Summary of the proposition-as-types principle

Textbook (pp. L4.11–12):

- (i) Propositions as types
- (ii) Proofs as programs
- (iii) Proof reduction as computation

References



Frank Pfenning (26th Jan. 2023). Lecture Notes on Proofs as Programs. Material for the course 15-317 Constructive Logic at Carnegie Mellon University, Spring 2023. URL: <https://www.cs.cmu.edu/~fp/courses/15317-s23/> (cit. on p. 2).



Morten-Heine Sørensen and Paul Urzyczyn (2006). Lectures on the Curry-Howard Isomorphism. Vol. 149. Studies in Logic and the Foundations of Mathematics. Elsevier (cit. on p. 12).